

A Generalisable Pipeline for Backporting ROCm to Obsolete AMD Hardware

Executive summary for technical mentorship

Soham Kulkarni

Core thesis

Build a reusable, benchmarked recovery pipeline for dropped AMD GPU support in ROCm, so older hardware can be revived without rediscovering the same failure modes from scratch.

Why this matters

Problem in one sentence

ROCm is the open-source route closest to CUDA for GPU compute.

AMD can drop older architectures from support.

Working hardware can become software-stranded.

What the project targets

dropped GPU support

fragmented community fixes

missing reproduction steps

unclear performance cost

The gap

Current reality

- scattered forum posts and issue threads
- ad hoc workarounds
- no single end-to-end reference
- little evidence about trade-offs

What is missing

- reproducible steps
- benchmarked comparisons
- clear failure boundaries
- a path others can reuse

Proposed approach: a decision tree for recovery



Key idea

The project is a systematic pipeline, not a one-off workaround. Each strategy isolates a different failure layer, and each successful configuration is measured.

Recovery strategies under test

Community baseline

spoofing and existing workarounds
how far unsupported hardware runs

Mixed-version stacks

ROCm userspace and AMDGPU kernel separation
whether failure is ABI, userspace, or architecture support

Containerised vendor stacks

AMD Docker images across ROCm versions
whether containers preserve a usable stack

Source builds

rebuild PyTorch and ROCm components for the target
whether compilation restores missing functionality

Start with the least invasive path, then escalate only when a step adds diagnostic value.

Scope: two representative targets, one reusable method

Initial hardware targets

gfx90c - Ryzen 5 5650U / Vega 7

gfx902 - Ryzen 5 2200G / Vega 8

Reference environments

Fedora 43 on the 5650U system

RHEL 9.5 on the 2200G desktop

ROCm 6.4.4 on the current host setup

Why these two

laptop and desktop form factors

same architectural family, different systems

cross-check for generalisation

Generalisation goal

prove the method on two targets first

reuse it for later dropped AMD architectures

How success will be measured

Evaluation at every stage

- direct HIP execution
- PyTorch runtime behavior
- performance relative to a working baseline
- reproducibility across systems and versions

Current test result

The current gfx900 spoof path for gfx90c reaches about 50% of AMD's published Vega 7 figure.

Comparison anchors

- latest stable ROCm for the target
- AMD's published figures
- the current workaround path

What has already been established

Post 1: context and motivation

ROCm ecosystem framing
hardware target selection
case for modern ML tooling on dropped
GCN5 hardware

Post 2: installation and blocker

native ROCm 6.4.4 on Fedora via DNF
metapackage
successful HIP execution
missing gfx90c kernels in bundled
rocBLAS for PyTorch

Status signal

The hardware compute stack is functional; the unresolved issue is support continuity in ROCm userspace and packaged kernels.

Execution path: from diagnosis to validation

1. Baseline

test existing workarounds

2. Decouple

mix ROCm userspace and
AMDGPU versions

3. Containerise

compare AMD Docker images across ROCm versions

4. Rebuild

compile PyTorch and needed
ROCm components

5. Validate

rerun on gfx902

6. Document

publish steps, benchmarks,
and failure modes

Why mentorship would matter

What I am trying to validate

dropped AMD GPU support can be recovered through spoofing, stack separation, containerisation, and selective source builds

benchmarks at every stage keep the work honest

Mentorship ask

I am asking for a GPU domain mentor with industry experience. I want practical guidance when I get stuck, and the FOSS United grants program requires a mentor.

Closing line

The goal is not just to make one GPU work; it is to build a method others can use when ROCm drops the next architecture.